Date

Parkly (Group 7)
Data Models Presentation

Tech Stack

Frontend

Next.js
→ Handles page rendering and **routing**
HTML / CSS
→ Structure and styling for UI

Backend

Node.js runtime
→ Runs our JS server

Express API framework
→ Defines API routes and handles requests

Database

PostgreSQL via Supabase
→ Stores structured data
ex: User profiles

Authentication (Part 1)

ENDPOINT MODULE

POST /auth/register

Core Purpose

Registers a new user. Validates that the user does not have an account, verifies that credentials are not taken, and hashes password for DB storage.

Functions Called

validateInputCredentials(userData: dict): status
hashPassword(pwd: string): string
storeUser(userData: dict): boolean

Testing

Unit tests: Call validateInputCredentials for users in the database, taken usernames, and users not in the database, check that it returns correctly. Verify that hashPassword follows bcrypt protocol.

Integration test: Verify that storeUser inserts into the DB correctly.

ENDPOINT MODULE

POST /auth/login

Core Purpose

Authenticates an existing user or tells user that they do not have an account. Issues JWT session token.

Functions Called

validateLogin(username: string, pwd: string): status
generateJWT(userID: string): string
getUserInfo(username: string): dict

Testing

Unit tests: Call validateLogin for correctly/incorrectly entered credentials and verify output. Validate generateJWT via verifyJWT.

Integration test: Verify GetUserInfo fetches the correct information from the DB.

Authentication (Part 2)

ENDPOINT MODULE

POST /auth/logout

Core Purpose

Logs out a user. Validates that the user's JWT is valid before removing the JWT to prevent further session usage.

Functions Called

verifyJWT(token: string): bool
verifyAuth(req: Request, res: Response, next: NextFunction): void
removeJWT(token: string): bool

Testing

Unit Tests: Call verifyJWT on JWTs that are valid/invalid tokens & verify output.

End to end test: Check that removeJWT prevents old JWTs from passing verifyJWT.

ENDPOINT MODULE

POST /auth/password-reset

Core Purpose

Handles any password change requests including forgot password and changes for validated users.

Functions Called

generateResetToken(userID: string): string
hashResetToken(token: string): string
getUserInfo(username: string): dict
sendResetLink(useInfo: dict, token: string): bool

Testing

Unit tests: Verify generation & hashing of reset tokens follows desired format. Mock API call for sendResetLink.

Integration test: Validate new password is correctly inserted into DB.

End-to-end testing of entire workflow

Authentication (Part 3)

ENDPOINT MODULE

GET /auth/iam

Core Purpose

Fetches user information for a logged in user. Removes/redacts sensitive fields as needed.

Functions Called

verifyJWT(token: string): bool
getUserInfo(username: string): dict
redactSensitiveData(userData: dict): dict

Testing

Unit test: Verify that sensitive information is redacted in the output dictionary of redactSensitiveData.

End-to-end testing of entire workflows

ENDPOINT INTEGRATION TEST

Register, Login, Logout, Password

Core Purpose

Common workflows for many users will involve registration, logging in, logging out, and changing passwords in some order. The purpose of this test is to ensure that the workflow runs smoothly for the user, database, and JWTs as needed.

Steps

1. User X, Password Y → login, expect: failure
2. Register User X, Password Y
3. User X, Password Y → login, expect: success
4. Run verifyJWT → expect: true
5. Log out , run verifyJWT → expect: false
6. Change password to Z while validated
7. User X, Password Y login → expect: failure
8. User X, Password Z login → expect: success
9. Repeat steps 6-8 but change password via email

Parking Spot Information

ENDPOINT MODULE

GET /listAvailableSpots

Core Purpose

Acts as a pipeline between our database and Next.js frontend.

Function Called

Calls spots.listAllAvailable(): spot[]

Queries all available spots from Supabase filtering status = ‘available’

Testing

Call spots.listAllAvailable() and compare the count to the count the DB returns for status = ‘available’

ENDPOINT MODULE

GET /listSpotsByZip?zip=[code]

Core Purpose

Provides location-based filtering to dynamically fetch spots

Function Called

Calls spots.searchByZip(zip: string): spot[]

Queries all available spots from Supabase filtering status = ‘available’ AND zip_code = zip

Testing

Call spots.searchByZip for various zip values and compare it with the count the DB returns.

Call with invalid zips to ensure it doesn’t crash

ENDPOINT MODULE

GET /getSpotDetails/:spotID=[id]

Core Purpose

Retrieves specific spot details when a user interacts with a spot.

Function Called

Calls spots.getSpotDetails(id: number): spot[]

Queries all Supbase for this specific spot and returns all the information of the spot.

Testing

Call spots.getSpotDetails() for various spots and compare the information returned to the true information.

Call with invalid IDs to ensure it doesn’t crash

Service Requests (Part 1)

ENDPOINT MODULE

GET /services

Core Purpose

Retrieves all active service requests associated with parked vehicles.

Functions Called

verifyJWT(token: string): bool
services.listRequests(): serviceRequest[]

Testing

Unit test:
Call listRequests() → verify only requests where status == "open" are returned.

Integration test:
Populate DB with open/closed requests → verify API returns correct open count.

ENDPOINT MODULE

POST /services

Core Purpose

Creates a new service request linked to a spotID or booking.

Functions Called

verifyJWT(token: string): bool
validateServiceRequest(data: object): status
services.createRequest(data: object): int

Testing

Unit test:
Call createRequest() with a mock request → verify function returns new request ID.

Integration test:
Submit "Car Wash" request for spotID 5 → verify row added to service_requests.

Service Requests (Part 2)

ENDPOINT MODULE

PATCH **/services/:requestID**

Core Purpose

Updates the status of a service request.

Functions Called

verifyJWT(token: string): bool
validateStatus(status: string): bool
services.updateRequestStatus(requestID: int, status: string): void

Testing

Unit test:
Call updateRequestStatus(12, "completed") → verify status value updates.

Integration test:
Send PATCH request → confirm DB status reflects "completed".

ENDPOINT MODULE

GET **/services/:requestID**

Core Purpose

Retrieves detailed information for a specific service request.

Functions Called

verifyJWT(token: string): bool
services.getRequestDetails(requestID: int): object

Testing

Unit test:
Call getRequestDetails(3) → verify returned object contains request data.

Integration test:
Query endpoint for known ID → verify response matches DB record.

Service Requests (Part 3)

ENDPOINT INTEGRATION TEST

Authenticate → Create → Update → Verify

Core Purpose

Tests the full service request lifecycle:
authentication, creation, status update, and database verification

Steps:

1. Send service request without token → verifyJWT expect: 401 Unauthorized
2. Send service request with valid JWT → verifyJWT expect: passes to next()
3. Call createRequest() with mock data for spotID 5 and type "Car Wash" → expect: new request ID returned
4. Check service_requests table → expect: new row inserted
5. Call updateRequestStatus(requestID, "completed") → expect: status updates successfully
6. Query getRequestDetails(requestID) → expect: returned object shows updated status

Payment (Part 1)

ENDPOINT MODULE

POST **/payment/process**

Core Purpose

Interfaces with a payment gateway and records the transaction in the database. Returns success or failure status.

Functions Called

processTransaction(data: dict): boolean
calculateTotal(spotID: int, hours: int): float

Testing

Unit test: Call calculateTotal(spotID, 3) for a $10/hr spot and 3 hours, verify function returns 30.00

Integration test: Submit a mock payment via processTransaction. Check the transactions table for a new success record.

ENDPOINT MODULE

POST **/middleware/auth**

Core Purpose

Express middleware that checks the Authorization header for a valid JWT and blocks unauthorized users before they reach any route handler.

Functions Called

verifyAuth(req: Request, res: Response, next: NextFunction): void
verifyJWT(token: string): boolean

Testing

Unit tests:
Send a request without a token. Verify the function returns a 401 Unauthorized response.

Send a request with a valid JWT. Verify next() is called and user reaches the route.

Payment (Part 2)

ENDPOINT INTEGRATION TEST

Calculate → Process → Verify

Core Purpose

A common payment workflow will involve verifying the user is authenticated, calculating the total cost, and processing the transaction. This test ensures the full flow runs correctly and that unauthenticated requests are blocked before reaching the payment logic

Steps:

1. Send payment request without token → verifyJWT expect: 401 Unauthorized
2. Send payment request with alid JWT → verifyJWT expect: passes to next()
3. Call calculateTotal(spotID, 3) for a $10/hr spot → expect 30.00
4. Call processTransaction with mock data → expect: True
5. Check transactions table → expect: new success record inserted
6. Call calculateToatal(spotID, 0) → expect: returns 0, no crash

Parking Provider Information (Part 1)

ENDPOINT MODULE

POST /addSpot

Core Purpose

Interacts with the parking_spots table to add a new spot with accompanying information like location, price, etc (returns spot ID)

Function Called

addSpot(details: { location: string; price: number; available: boolean; providerID: number; }): number

Adds a spot with location, price, availability into Supabase Table parking_spots

Testing

Submit a spot at "123 Main St". Verify id is returned and stored in the DBMS

ENDPOINT MODULE

PATCH /updateAvailability?spotID=[id]

Core Purpose

Toggles the “available” column for a specific spot, making it disappear from the booking feed

Function Called

updateAvailability(spotID: number, status: boolean): void

Updates the specific parking ID to available or unavailable

Testing

Change status to false. Attempt to retrieve spot via /feed; it should no longer appear

ENDPOINT MODULE

POST /deleteSpot?spotID=[id]

Core Purpose

Removes the record from PostgreSQL parking_spots table.

Function Called

deleteSpot(spotID: number): string

Testing

Call deleteSpot(id=10). Verify via Supabase that row 10 is gone

Call with invalid IDs to ensure it doesn’t crash

Parking Provider Info Part 2

ENDPOINT MODULE

GET **/getProviderHistory**

Core Purpose

Find all spots that an Owner has provided in the past, owner_id == provider_id

Function Called

getProviderHistory(providerID: number): Spot[]

Testing

Populate 3 spots for User A. Ensure function returns an array of exactly 3 objects.

Call with invalid IDs to ensure it doesn’t crash

Thank you!